

ODI Demo Build — 1-Hour Live Session with Claude

Goal: stand up a working ODI / Managed Data Lake demo and get as far as time allows. Boss is hands-on; you pre-stage the infra.

Before the call — your pre-flight (this is what makes 1 hour viable)

- 1. export FIVETRAN_API_KEY=key:secret and confirm a call to /v1/groups returns 200.
- 2. Apply the Terraform in infra/ (S3 bucket + Glue bronze/silver/gold + IAM role). Confirm terraform output shows **bucket_name** and **fivetran_role_arn**.
- 3. Verify the Fivetran **external_id** + AWS account id in tfvars are current — if stale, the destination create fails on permissions. Fix it now, not on stage.
- 4. Have an **OAuth source account** ready (Salesforce, HubSpot, or Google) that you can sign into from this machine. SSO orgs can add a hop — test the sign-in beforehand.
- 5. For the dbt stretch: confirm the dbt-athena env vars (LAKE_BUCKET, ATHENA_WORKGROUP) and that the Athena workgroup exists. Node/npm installed for the frontend stretch.
- 6. Do one full dry run yourself, then delete that throwaway group so the live account is clean. Keep the Fivetran dashboard and the AWS Glue/Athena console open in browser tabs.

Lock these before you start

- **Vertical / industry** (drives the demo story and the dbt + frontend content)
- **Org name** — Niraj rule: used in every title, H1, and navbar; pick a generic name that won't collide with a real entity
- **Buyer persona**
- **OAuth source + which tables to sync** (keep it small so the first sync is fast)

The hour — priority order (CORE first, STRETCH if time remains)

0:00	Kickoff: invoke the skill, confirm vertical/persona/org + that infra is staged	Both	
0:05	GATHER: choose the OAuth source and the tables to sync	Boss	
0:08	Prompt 1 — create the AWS MDLS destination	Boss	CORE
0:14	Prompt 2 — create connector + open Connect Card, sign in via OAuth	Boss	CORE
0:24	Prompt 3 — run the setup test (expect CONNECTED)	Boss	CORE
0:27	Prompt 4 — trigger the first sync (runs in background)	Boss	CORE
0:30	While syncing: Hybrid-deployment talking point + confirm infra	You	
0:35	Prompt 5 — verify tables + rows in the Iceberg lake = MIN DEMO DONE	Boss	CORE
0:42	Prompt 6 — scaffold dbt bronze/silver/gold (+ Niraj rules)	Boss	STRETCH
0:50	Prompt 7 — dbt build (gold Iceberg tables) = GOOD DEMO DONE	Boss	STRETCH
0:55	Prompt 8 — React frontend shell + frontend-design pass	Boss	STRETCH
0:58	Wrap: what's done, what's a follow-up	You	

The prompts he types (paste one at a time; let Claude finish each)

Kickoff — start the build

Let's build a new ODI demo with the odi-demo-builder skill. Vertical: [VERTICAL]. Org name: [ORG]. Buyer persona: [PERSONA]. The Terraform in ./infra is already applied and \$FIVETRAN_API_KEY is set — confirm the terraform outputs and the API connection, then take me through PROVISION and RUN, and as much of BUILD as we have time for.

GATHER — pick the source

Use a [Salesforce / HubSpot / Google] OAuth source for this demo. Sync these tables: [list a few]. Keep the selection small so the first historical sync finishes quickly.

Prompt 1 — AWS MDL S destination (CORE)

Create the Fivetran Managed Data Lake destination on AWS now. Make a new group called boss-demo. Run terraform output in ./infra for the S3 bucket, IAM role ARN, and Glue bronze database, and use those in us-east-1. Call the REST API with the key from \$FIVETRAN_API_KEY, and show me the destination id and setup status.

Prompt 2 — OAuth connector (CORE)

Create the [source] connector in the boss-demo group via the REST API, then generate a Fivetran Connect Card authorization link and give me the URL so I can sign in with OAuth.

In the browser: open the Connect Card URL Claude returns, sign in with OAuth, pick the data to sync, click Save. No passwords typed.

Prompt 3 — test the connector (CORE)

Run the connector setup test and tell me whether it reports CONNECTED. If it's INCOMPLETE, surface the failure.

Prompt 4 — trigger the sync (CORE)

Trigger the first sync on that connector, then poll sync_state every 30 seconds until it's SYNCED. Tell me when it's done and which tables landed.

Prompt 5 — verify the lake (CORE)

List the S3 bronze prefix, then query the tables through Athena against the Glue catalog and show me a sample of rows from the largest table, so we confirm the data actually landed in the Iceberg lake.

Checkpoint — minimum demo done

Live OAuth connector + first sync landed as Iceberg tables in the lake. That's a real ODI spine you can demo. Everything below is upside.

Prompt 6 — dbt scaffold (STRETCH)

Scaffold the dbt project to the portfolio standard: bronze sources.yml from the synced tables, silver staging models, and one gold Iceberg fact/dimension. Apply Niraj's content rules — org name [ORG] everywhere, every connector gets its Fivetran deep link, no raw table names exposed.

Prompt 7 — dbt build (STRETCH)

Run dbt build --select bronze silver gold and confirm the gold Iceberg tables are created in the Glue catalog. If it fails, diagnose from the error before fixing.

Checkpoint — good demo done

Data flowing through a bronze/silver/gold dbt pipeline into gold Iceberg tables. Strong stopping point.

Prompt 8 — frontend shell (STRETCH)

Generate the React frontend shell — a Landing page with the [ORG] hero and a pipeline diagram (Fivetran -> S3/Iceberg -> dbt -> Athena), and a Pipeline page listing the connector with its Fivetran deep link. Then run the frontend-design pass for the [VERTICAL] vertical and [PERSONA] buyer.

After the demo — cleanup

If boss-demo was a throwaway, tear down its group, connector, destination, and the Terraform resources so the account and AWS are clean. If we're keeping this as the real demo, leave it and just confirm what's deployed.

Keep the demo if it's the deliverable; only tear down throwaways. Don't delete the staged infra you'll reuse.

While the sync runs — show the range (you drive)

- "Research [account] via the FivetranKnowledge MCP — open tickets, last Gong call."
- "Draft a QBR deck outline for that account."
- Hybrid deployment aside: the agent runs in the customer's network, only metadata leaves their perimeter — same connector, destination, and pipeline. Answers the data-residency objection.